

关于信号量的说法正确的是 ()

- A 信号量的初值可以为负数
- B 信号量的值定义后无法修改
- C 若有一信号量 s ，可以执行 $s=s-1$
- D 信号量的值可以执行P、V操作修改

关于信号量的PV操作描述不正确的是 ()。

- ☒ A 进程调用P操作后可能会阻塞该进程自身
- ☒ B 进程调用P操作后会阻塞另一个进程
- ☒ C 进程调用V操作后不会影响进程自身的执行
- ☒ D 进程调用V操作后可能影响到另一个进程的状态

用V操作唤醒一个等待进程时，被唤醒进程的状态变为（ ）。

- ☐ A 等待
- ☒ B 就绪
- ☐ C 运行
- ☐ D 完成



单选题 1分

用P、V操作管理相关进程的临界区时，信号量的初值应定义为（ ）

- A -1
- B 0
- ☒ C 1
- D 任意值

单选题 1分

若两个并发进程相关临界区的互斥信号量MUTEX现在取值为0，则正确的描述应该是（）。

- ☐ A 没有进程进入临界区
- ☒ B 有一个进程进入临界区
- ☐ C 有一个进程进入临界区，另一个在等待进入临界区
- ☐ D 不定

多选题 2分

若记录型信号量S的初值是3，则当前值为1时表示（ ）

- ☐ A 系统对应阻塞等待队列存在1个进程
- ☐ B 系统对应阻塞等待队列存在2个进程
- ☒ C 系统对应阻塞等待队列存在0个进程
- ☒ D 系统现有对应可用资源1个
- ☒ E 系统现有对应可用资源2个

Part 2 进程同步机制

2.4 管程机制

问题

- 需要程序员实现，编程困难
- 维护困难
- 容易出错
 - wait/signal位置错
 - wait/signal不配对

解决方法

- 由**编程语言**解决同步互斥问题
- 管程 (1970s, Hoare 和Hansen)

信号量: **分散式**
管 程: **集中式**

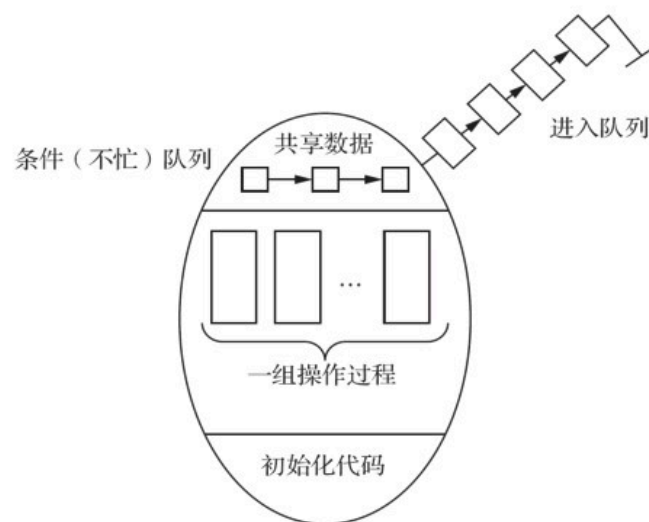
Part 2 进程同步机制

2.4 管程机制

管程定义



- 一个管程定义了一个**数据结构**和能为并发进程所执行（在该数据结构上）的**一组操作**，这组操作能同步进程和改变管程中的数据



Part 2 进程同步机制

2.4 管程机制



语法描述如下:

```
Monitor monitor_name {           /*管程名*/
    share variable declarations; /*共享变量说明*/
    cond declarations;           /*条件变量说明*/
    public:                      /*能被进程调用的过程*/
        void P1(.....) {.....} /*对数据结构操作的过程*/
        void P2(.....) {.....}
        .....
        void (.....) {.....}
    .....
    {                             /*管程主体*/
        initialization code;      /*初始化代码*/
        .....
    } }
```

Part 2 进程同步机制

2.4 管程机制

互斥

- 管程中的变量只能被管程中的操作访问
- 任何时候只有一个进程在管程中操作
- 类似临界区
- 由编译器完成

同步

- 条件变量
- 唤醒和阻塞操作

Part 2 进程同步机制

2.4 管程机制

条件变量



condition x, y;



条件变量的操作

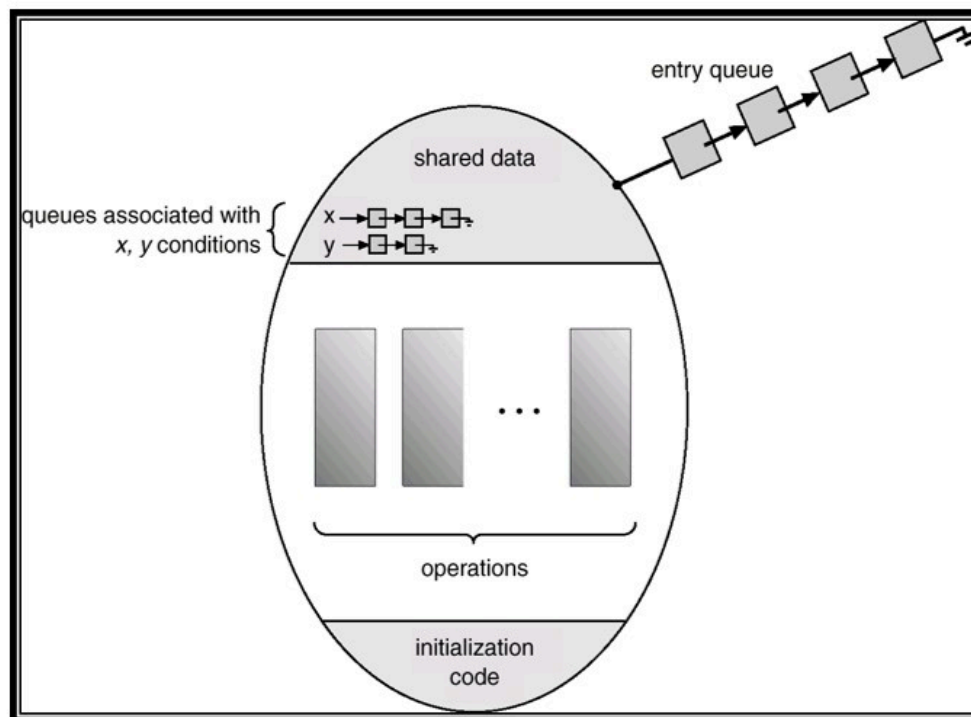
- 阻塞操作: wait
- 唤醒操作: signal



x.wait(): 进程阻塞直到另外一个进程调用x.signal()



x.signal(): 唤醒另外一个进程



Part 2 进程同步机制

2.4 管程机制

条件变量问题

管程内可能存在不止1个进程。

➤例如：进程P调用signal操作唤醒进程Q后。

存在的可能处理方式：

- P等待，直到Q离开管程或等待另一条件 (Hoare) 。
- Q等待，直到P离开管程或等待另一条件 (Hansen) 。

Part 3 经典进程同步问题



学习目标

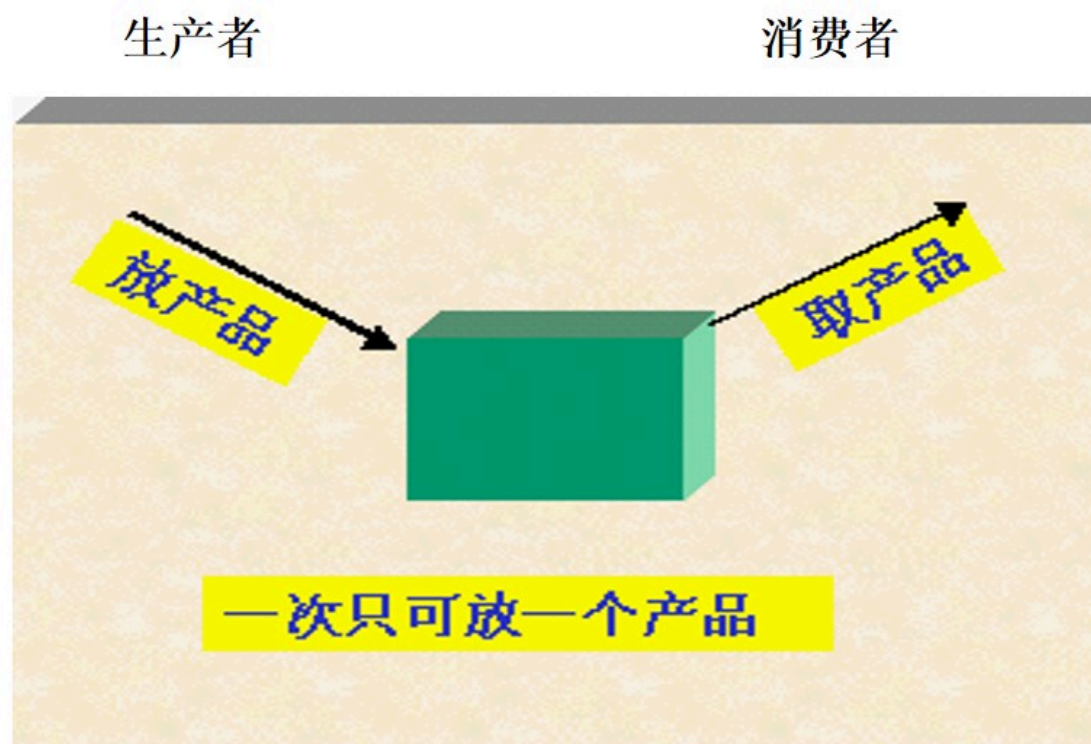
- 能够分析生产者消费者问题的解决方案并应用
- 能够理解并分析读者写者问题的解决方案
- 能够分析哲学家进餐问题的解决方案

Part 3 经典进程同步问题

3.1 生产者-消费者问题



● 生产者 - 消费者问题



Part 3 经典进程同步问题

3.1 生产者-消费者问题

- 1. 一个生产者，一个消费者，一个缓冲区

semaphore S1,S2; S1.value=0;2.value=0;

- 生产者

While(true)

{ 生产一件产品;

P(s1)

放入一件产品;

V(s2)

}

- 消费者

While(true)

{ **P(s2)**

取出一件产品;

V(s1)

消费产品;

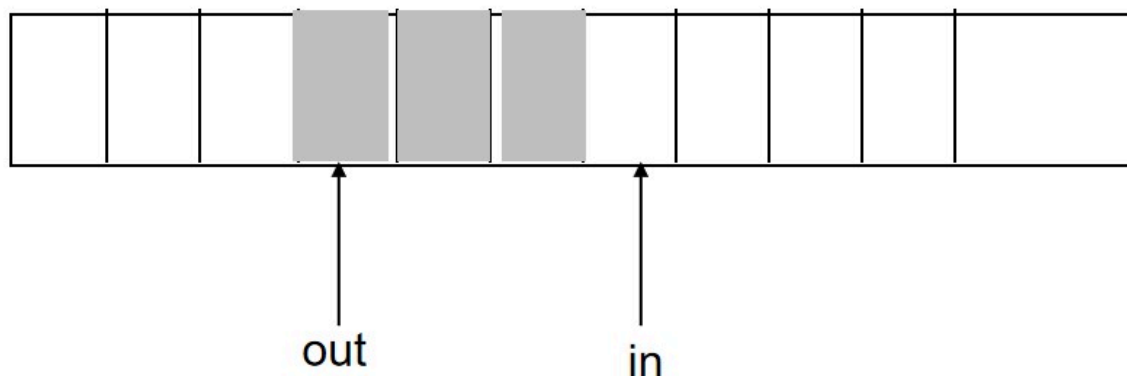
}

Part 3 经典进程同步问题

3.1 生产者-消费者问题

- 2. 多个生产者，多个消费者，n个缓冲区

1 2 3 4 5 6 7 8 9 10



● Q: 两个生产者进程同时向缓冲区中放产品?

Part 3 经典进程同步问题

3.1 生产者-消费者问题

临界资源
生产者



➤ 把产品放入指定缓冲区

➤ **in**: 所有的生产者对 **in** 指针需要互斥

➤ **counter**: 所有生产者消费者进程对 **counter** 互斥

```
buffer[in] = nextp;  
in = (in + 1) % N;  
counter++;
```

生产者消费者的互斥分析

消费者

➤ 从指定缓冲区取出产品

➤ **out**: 所有的消费者对 **out** 指针需要互斥

➤ **counter**: 所有生产者消费者进程对 **counter** 互斥

```
nextc = buffer[out];  
out = (out + 1) % N;  
counter--;
```

Part 3 经典进程同步问题

3.1 生产者-消费者问题

```
semaphore empty,full; empty.value=n,full.value=0;  
semaphore mutex; mutex.value=1;
```

```
void producer(){  
    While(true)  
    { 生产一件产品;  
      wait(empty)  
      wait(mutex)  
      放入一件产品;  
      signal(mutex)  
      signal(full)  
    }  
}
```

Q: wait(empty),
wait(mutex)
互换?

Q: signal
(mutex)和
signal (full)
互换?

```
void consumer(){  
    While(true)  
    { wait(full)  
      wait(mutex)  
      取出一件产品;  
      signal(mutex)  
      signal(empty)  
      消费产品;  
    }  
}
```

Part 3 经典进程同步问题

3.1 生产者-消费者问题

● 3.利用AND信号量解决

```
int in=0, out=0;
item buffer[n];
semaphore mutex.value=1,
empty.value=n, full.value=0;
void producer()
{ do{
    produce an item in nextp;
    Swait(empty, mutex);
    buffer(in) =nextp;
    in =(in+1)mod n;
    Ssignal(mutex, full);
} while(TRUE);
}1
```

```
void consumer(){
    do {
        Swait(full, mutex);
        nextc=buffer(out);
        out=(out+1) mod n;
        Ssignal(mutex, empty);
        consumer the item ;
    }while(TRUE);
}
```

Part 3 经典进程同步问题

3.1 生产者-消费者问题

4. 利用管程解决

```
void get(item x) {  
    if (count <= 0)  
        cwait(notempty);  
    x = buffer[out];  
    out = (out+1) % N;  
    count--;  
    csignal(notfull);  
}  
{ in=0; out=0; count=0;  
}  
}PC;
```

```
Monitor producerconsumer {  
    item buffer[N];  
    int in, out;  
    condition notfull, notempty;  
    int count;  
public:  
    void put(item x) {  
        if (count >= N) cwait(notfull);  
        buffer[in] = x;  
        in = (in+1) % N;  
        count++;  
        csignal(notempty);  
    }  
}
```

Part 3 经典进程同步问题

3.1 生产者-消费者问题

```
void producer() {  
    item x;  
    while(TRUE) {  
        .....  
        produce an item in  
        nextp;  
        PC.put(x);  
    }  
}
```



```
void consumer() {  
    item x;  
    while(TRUE) {  
        PC.get(x);  
        consume the item  
        .....  
    } }  
  
void main() {  
    cobegin  
        proceducer(); consumer();  
    coend  
}
```

Part 3 经典进程同步问题

3.2 读者-写者问题

有两组并发进程: **要求:**

- 读者和写者, 共享一组数据区。
- 允许多个读者同时执行读操作;
- 不允许读者、写者同时操作;
- 不允许多个写者同时操作。

分类:

- 读者优先 (第一类读者写者问题)
- 写者优先 (第二类读者写者问题)

解决方案:

- 记录型信号量
- 信号量集

Part 3 经典进程同步问题

3.2 读者-写者问题

❖ w 读写互斥信号量，初值是1

semaphore w=1;

```
void reader() {  
    do {  
        P(w);  
        读  
        V(w);  
    }  
}while(TRUE)
```

● Q:两个读者进程同时读?

```
void writer() {  
    do {  
        P(w);  
        写  
        V(w);  
    }  
}while(TRUE)
```

Part 3 经典进程同步问题

3.2 读者-写者问题

❖ w 读写互斥信号量，初值是1, n是读者计数的整数值

semaphore w=1;

```
void reader() {  
    do {  
        if (n==0) P(w);  
        n++;  
        读  
        n--;  
        if (n==0) V(w);  
    }  
    }while(TRUE)
```

● Q: 两个读者进程同时读?

```
void writer() {  
    do {  
        P(w);  
        写  
        V(w);  
    }  
    }while(TRUE)
```

Part 3 经典进程同步问题

3.2 读者-写者问题

- ❖ readcount 对读者计数，初值是0；
- ❖ mutex表示对readcount的互斥操作，初值是1。

Part 3 经典进程同步问题

3.2 读者-写者问题

初始化:

semaphore

mutex=1, w=1;

int readcount=0;

void **writer**() {

do {

wait(w);

写

signal(w);

}

}while(TRUE)

void **reader**() {

do {

wait(mutex);

if (readcount==0) wait(w);

readcount++;

signal(mutex);

读

wait(mutex);

readcount--;

if (readcount==0) signal(w);

signal(mutex);

}while(TRUE);

}

Part 3 经典进程同步问题

3.2 读者-写者问题

写者优先(第二类读者写者问题)



问题描述

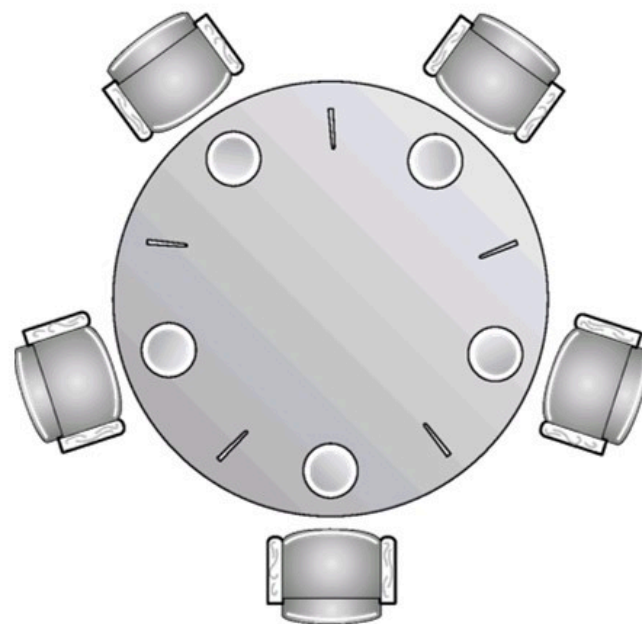
- 多个读者可以同时进行读
- 写者必须互斥（只允许一个写者写，也不能读者写者同时进行）
- 写者优先于读者（一旦有写者，则后续读者必须等待，唤醒时优先考虑写者）

如何用PV操作实现？（思考题）

Part 3 经典进程同步问题

3.3 哲学家进餐问题

- 五个哲学家的生活方式：交替思考和进餐共用一张圆桌，分别坐在五张椅子上在圆桌上有五个碗和五支筷子
- 平时哲学家思考，饥饿时便试图取用其左、右最靠近他的筷子，只有在拿到两支筷子时才能进餐进餐毕，放下筷子又继续思考



Part 3 经典进程同步问题

3.3 哲学家进餐问题

Semaphore chopstick[5] = {1,1,1,1,1};

Philosopher i:

do {

wait(chopStick[i]);

wait(chopStick[(i + 1) % 5]);

eat for awhile

signal(chopStick[i]);

signal(chopStick[(i + 1) % 5]);

think for awhile

} while (true)

●Q:有没有问题?

Part 3 经典进程同步问题

3.3 哲学家进餐问题



可能引起死锁，如五个哲学家同时饥饿而各自拿起左筷子时，会使信号量chopstick均为0；因此他们试图去拿右筷子时，无法拿到而无限期等待。



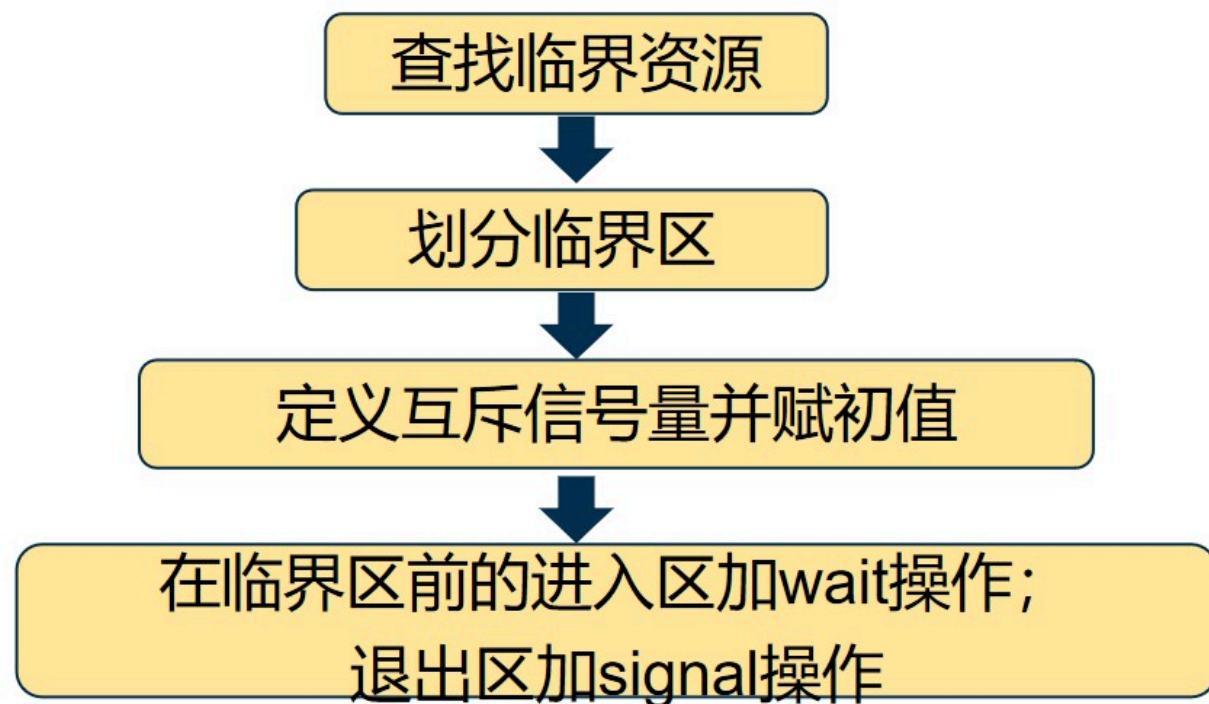
解决方法：

- ① 最多允许4个哲学家同时坐在桌子周围
- ② 仅当一个哲学家左右两边的筷子都可用时，才允许他拿筷子。
- ③ 给所有哲学家编号，奇数号的哲学家必须首先拿左边的筷子，偶数号的哲学家则反之

Part 3 经典进程同步问题

3.4 信号量使用总结

互斥分析基本方法



Part 3 经典进程同步问题

3.4 信号量使用总结

关于PV的操作讨论



PV操作的使用

- P.V操作必须成对出现，有一个P操作就一定有一个V操作
- 当为互斥操作时，它们同处于同一进程
- 当为同步操作时，则不在同一进程中出现

进程管理总结

进程控制：进程定义、状态、特征、组成、控制（创建、终止、阻塞、唤醒）、线程

进程通信：信号、管道、消息队列、共享内存

进程调度：作业调度、进程调度（调度算法）

进程同步：同步互斥、软件、硬件信号量、管程、经典同步问题